
Problem 8.22.

Part a. Let $ADD = \{\langle x, y, z\rangle \mid x, y, z > 0 \text{ are binary integers and } x + y = z\}$.
Show that $ADD \in$ L.

Proof. To show that $ADD \in$ L, we give a log space algorithm M that decides ADD.

$M =$ "On input $\langle x, y, z\rangle$, where x, y and z are binary integers greater than 0:

1. Let i, n_x, n_y, n_z and m be binary integers, and s and c be binary digits on work tape.
2. Let $n_x = |x|$, $n_y = |y|$, $n_z = |z|$, and m be maximum of n_x and n_y.
3. Let $i = 1$, and $c = 0$.
4. Repeat for m steps:
5. If z does not have i^{th} digit ($i > n_z$), then *reject*.
6. Let s be the sum of c and i^{th} binary digits of x and y. If either x or y does not have i^{th} binary digit, then assume the digit to be 0.
7. If the addition produces a carry, then $c = 1$. Otherwise, $c = 0$.
8. If s and i^{th} digit of z are different, then *reject*.
9. Increment i by 1.
10. If $c = 1$, then:
11. If z does not have i^{th} digit, then *reject*.
12. If c and i^{th} digit of z are different, then *reject*.
13. If z has $(i + 1)^{th}$ digit, then *reject*.
14. Otherwise, if z has i^{th} digit, then *reject*.
15. *Accept*."

□

Part b. Let $PAL-ADD = \{\langle x, y\rangle \mid x, y > 0$ are binary integers where $x+y$ is an integer whose binary representation is a palindrome$\}$. (Note that the binary representation of the sum is assumed not to have leading zeros. A palindrome is a string that equals its reverse.)
Show that $PAL - ADD \in$ L.

Proof. To show that $PAL-ADD \in$ L, we give a log space algorithm M that decides $PAL-ADD$.

$M =$ "On input $\langle x, y\rangle$, where x and y are binary integers greater than 0:

1. Let i, j and m be binary integers on work tape.
2. Let m be the maximum of $|x|$ and $|y|$.
3. Run C on input $\langle x, y\rangle$. If C accepts then $m = m + 1$.
4. Let $i = 1$, and $j = m$.
5. Repeat for $m - 1$ steps:
6. Run B on $\langle x, y, i\rangle$. If B accepts, then $b_i = 1$. Otherwise $b_i = 0$.
7. Run B on $\langle x, y, j\rangle$. If B accepts, then $b_j = 1$. Otherwise $b_j = 0$.
8. If b_i and b_j are different, then *reject*.
9. Increment i by 1, and decrement j by 1.
10. *Accept*."

Let $CARRY = \{\langle x, y\rangle \mid x, y > 0$ are binary integers whose sum produces a carry$\}$. The algorithm C decides $CARRY$ in log space.

$C =$ "On input $\langle x, y\rangle$, where x and y are binary integers greater than 0:

1. Let i and m be binary integers on work tape where $i = 1$ and $m = max(n_x, n_y)$.
2. Let s and c be binary digits on work tape where $s,\ c = 0$.
3. Repeat for m steps:
4. Let s be the sum of c and i^{th} binary digits of x and y. If either x or y does not have i^{th} binary digit, then the digit is 0.
5. If the addition produces a carry, then $c = 1$. Otherwise, $c = 0$.
6. Increment i by 1.
7. If $c = 1$ then *accept*. Otherwise, *reject*."

Let $BIT = \{\langle x, y, i\rangle \mid x, y, i > 0$ are binary integers, and i^{th} digit of $x + y$ is $1\}$. The algorithm B decides BIT in log space.

$M =$ "On input $\langle x, y\rangle$, where x and y are binary integers greater than 0:

1. Let i, j, k, n_x, n_y, m and n be binary integers, and s, c, b_i and b_j be binary digits on work tape.
2. Let $n_x = |x|$, $n_y = |y|$, and m be maximum of n_x and n_y.
3. Run C on input $\langle x, y\rangle$. If C accepts then $n = m + 1$. Otherwise $n = m$.
4. Let $i = 1$, and $j = n$.
5. Repeat for n steps:
6. Let $k = 1$, and $c = 0$.
7. Repeat for m steps:
8. Let s be the sum of c and k^{th} binary digits of x and y. If either x or y does not have k^{th} binary digit, then assume the digit to be 0.
9. If the addition produces a carry, then $c = 1$. Otherwise, $c = 0$.
10. If $i = k$, then $b_i = s$.
11. If $j = k$, then $b_j = s$.
12. Increment j by 1.
13. If $c = 1$ Repeat for m steps:
14. *Accept*."

□